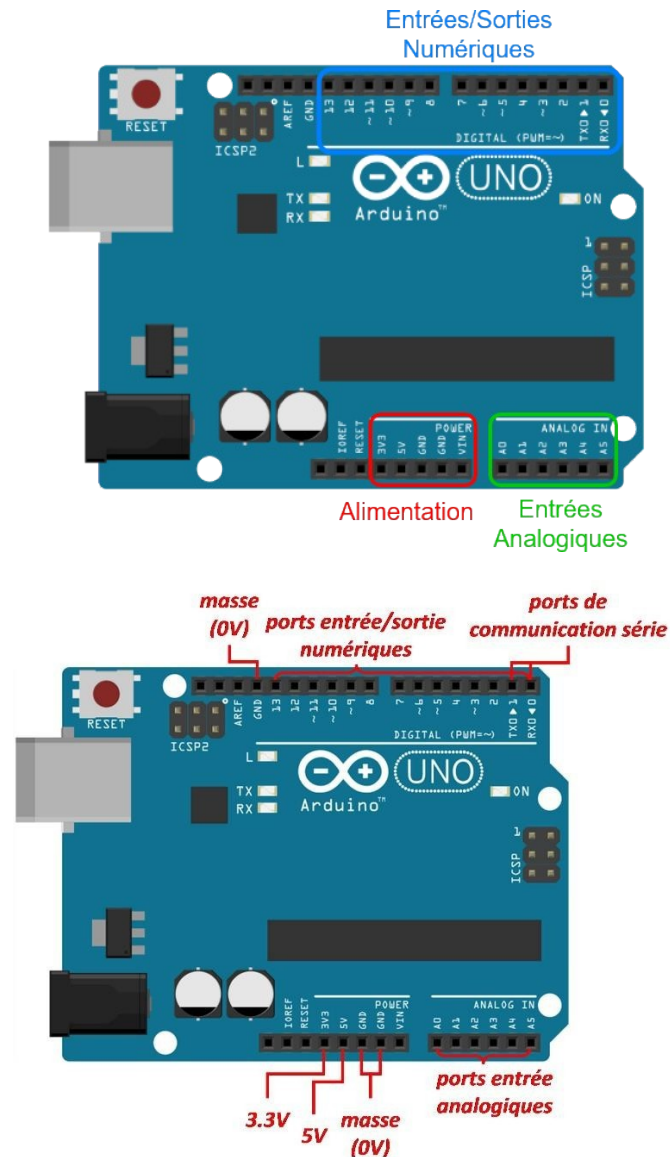


TD N°1 : Initiation à la programmation Arduino

Description de la carte Arduino



- Les ports numériques (0 à 13)



Ils sont au nombre de 14 (Arduino UNO), et peuvent être configurés comme des entrées, ou bien comme des sorties.

En mode « sortie »

Configuration :

```
pinMode(8, OUTPUT);
```

Utilisation en mode *tout ou rien* : 0V ou 5V

ou en mode analogique (par génération d'un signal [PWM](#)). !!

En mode « entrée »

Configuration :

```
pinMode(8, INPUT);
```

Selon la tension appliquée à leur connecteur, ils commutent entre un état HAUT et un état BAS .

Les ports analogiques (A0 à A5)



Ils sont au nombre de 6 (Arduino UNO) et sont tous des ports d'entrée.

Ils convertissent la valeur de la tension appliquée à leur connecteur (0V à 5V) en un nombre (entre 0 et 1023, soit 10 bits)

Exercice 1 :

```
1  // Module IOT - Filière SI - S5
2  // Exercice N°1
3  int led = 13;
4
5  void setup() {
6      pinMode(led, OUTPUT);
7  }
8
9  void loop() {
10     digitalWrite(led, HIGH);
11     delay(1000);
12     digitalWrite(led, LOW);
13     delay(1000);
14 }
```

1. Identifier les parties principales du programme.

Le programme Arduino comporte des parties principales :

- `Setup ()` :

Cette fonction s'exécute une seule fois au démarrage de la carte

Elle sert à configurer les broches ou à initialiser des éléments

- `Loop()` :

Cette fonction s'exécute en boucle infinie

Elle contient les instructions que la carte Arduino répète sans arrêt

2. Expliquer le rôle de chaque fonction utilisée.

`pinMode(Led, OUTPUT)` : Configure une broche comme sortie numérique

`digitalWrite(Led, HIGH)` : Met la broche à l'état haut (+5V) pour allumer la LED

`digitalWrite(Led, LOW)` : Met la broche à l'état bas (0V) pour éteindre la LED

`delay(1000)` : Met le programme en pause pendant le temps spécifié en millisecondes

`setup()` : Fonction d'initialisation exécutée une fois au démarrage

`loop()` : Fonction principale exécutée en boucle infinie après le `setup`

3. Que se passe-t-il si on remplace `delay(1000)` par `delay(200)` ?

Avec `delay(1000)` :

- La LED clignote toutes les **2 secondes** (1s allumée + 1s éteinte)

- Fréquence de clignotement : **0,5 Hz**

Avec `delay(200)` :

- La LED clignote toutes les **0,4 secondes** (0,2s allumée + 0,2s éteinte)

- Fréquence de clignotement : **2,5 Hz**

- **La LED clignote 5 fois plus vite !**

4. Ajouter un commentaire clair à chaque ligne.

Le changement de `delay(1000)` à `delay(200)` fait clignoter la LED 5 fois plus rapidement, passant d'un clignotement lent et visible à un clignotement rapide et plus nerveux.

Exercice 2

Reprenez le code du **clignotement de la LED**, mais :

- Utilisez une **autre broche** (par exemple D4 ou 2 selon la carte) ;
- Changez la fréquence de clignotement ;
- Ajoutez un **deuxième clignotement rapide** avant chaque pause longue.

Exemple attendu : la LED clignote deux fois rapidement, puis fait une pause d'une seconde.

- Faire en sorte que la durée de clignotement soit réglable via une variable *delai*.

```
int led = D4;
```

```
int delai = 500;
```

```
void setup() {  
  pinMode(led, OUTPUT);  
}
```

```
void loop() {  
  digitalWrite(led, HIGH);  
  delay(delai / 4);  
  digitalWrite(led, LOW);  
  delay(delai / 4);
```

```
  digitalWrite(led, HIGH);  
  delay(delai / 4);  
  digitalWrite(led, LOW);  
  delay(delai / 4);
```

```
  digitalWrite(led, HIGH);  
  delay(delai * 2);  
  digitalWrite(led, LOW);  
  delay(delai * 2);
```

```
}
```

Exercice 3 :

On dispose :

- d'un **bouton poussoir** branché sur la broche D2,
- d'une **LED** sur la broche D4.

Écrire un programme qui :

1. Lit l'état du bouton (0 ou 1).
2. Allume la LED quand le bouton est **appuyé**.
3. Éteint la LED sinon.
4. Ajouter une **condition inverse** avec INPUT_PULLUP (le bouton actif à l'état bas).

```
int bouton = 2;
int led = 4;
void setup() {
  pinMode(bouton, INPUT);
  pinMode(led, OUTPUT);

  digitalWrite(led, LOW);
}

void loop() {
  // Lecture de l'état du bouton
  int etatBouton = digitalRead(bouton);

  if (etatBouton == HIGH) {
    digitalWrite(led, HIGH);
  } else {
    digitalWrite(led, LOW);
  }
  delay(10);
}
```

Exercice 4

```
1  // Module IOT - Filière SI - S5
2  // Exercice N°4 - ESP
3  #include <ESP8266WiFi.h>
4
5  void setup() {
6      Serial.begin(115200);
7      WiFi.mode(WIFI_STA);
8      WiFi.disconnect();
9      delay(100);
10     Serial.println("Scan des réseaux WiFi disponibles...");
11 }
12
13 void loop() {
14     int n = WiFi.scanNetworks();
15     Serial.println("Scan terminé !");
16     if (n == 0) {
17         Serial.println("Aucun réseau trouvé");
18     } else {
19         Serial.print(n);
20         Serial.println(" réseaux trouvés :");
21         for (int i = 0; i < n; ++i) {
22             Serial.print(i + 1);
23             Serial.print(": ");
24             Serial.print(WiFi.SSID(i));
25             Serial.print(" (signal ");
26             Serial.print(WiFi.RSSI(i));
27             Serial.println(" dBm)");
28             delay(10);
29         }
30     }
31     delay(5000);
32 }
```

1. Identifier la **bibliothèque utilisée** et son rôle.

Bibliothèque :

```
#include <ESP8266WiFi.h>
```

Rôle :

Cette bibliothèque permet d'utiliser les fonctions WiFi de l'ESP8266.

Elle contient toutes les méthodes nécessaires pour :

se connecter à un réseau WiFi,

créer un point d'accès,

scanner les réseaux WiFi à proximité,

obtenir des informations comme le SSID, le signal, etc.

2. Expliquer la fonction *WiFi.scanNetworks()*.

Fonctionnement :

Elle recherche les signaux WiFi présents

Elle retourne un nombre entier n, qui correspond au nombre de réseaux détectés.

Chaque réseau peut ensuite être identifié par son index (i) : SSID(i) pour le nom, RSSI(i) pour la puissance.

3. Que représentent *SSID(i)* et *RSSI(i)* ?

Element	Signification	Explication
SSID(i)	Servuce Set Identifier	C'est le nom du réseau WiFi détecté.
RSSI(i)	Received Signal Strength Indicator	C'est la puissance du signal reçue (en dBm). Plus la valeur est proche de 0, plus le signal est fort.